

Instructions for Building SIMLA

August 11, 2025

1 Description of Database Tables

1.1 `bcd_metadata`

This table is used to quickly access basic metadata for BCDs without having to open the FITS files. Only data that can be read directly from the header belong in this table, excluding the `FILE_NAME`. Each row corresponds to a unique DCEID.

Script: `/simlapath/database_build/bcd_metadata.py`

Columns:

- `DCEID`: (int) code that is unique for each BCD. There are 878269 BCDs in total.
- `FILE_NAME`: (str) for the actual bcd file path. Begins after the set `irspath`, so it assumes that the IRS data directory structure has not been changed.
- `AORKEY`: (int) code that is unique for each “observation area” at a certain time. These are what we use to group cubes, so one AOR = one cube (per suborder).
- `CHNLNUM`: (int). SL, SH, LL, LH = 0, 1, 2, 3.
- `CAMPAIGN`: (str) code unique to each observation campaign.
- `PROGID`: (int) code unique for each program ID (program = proposal).
- `OBJECT`: (str) PI-input name for the object of interest.
- `OBJTYPE`: (str) that can be used to filter out multi-target or moving observations.
- `RA_FOV`: (float) coordinate at the FOV averaged over the slit, but there are many FOV types. These could be used out of the database for estimated positions, but for accurate slit positions, use the `irs_slit_positions` code.

- DEC_FOV: (float) see above.
- PA_FOV: (float) position angle, see RA_FOV.
- MJD_OBS: (float) for the modified julian date of the observation.
- RAMPTIME: (float) for the total integration time, there are only a few options for each CHNLNUM. In general, BCDs with different RAMPTIMEs should not be used together.
- SAMPTIME: (float) for the integration time of each “sample”.
- AOT_TYPE: (str) for the template type. Used to distinguish maps from staring mode observations.
- FOVNAME: (str) distinguishing the reference point for an observation.
- FOVID: (int) code for the above.
- STEPSPAR: (int) the number of slit steps in the parallel direction.
- STEPSPER: (int) the number of slit steps in the perpendicular direction.

There are indices on DCEID, AORKEY, CHNLNUM, MJD_OBS, and RAMPTIME (`bcd_metadata_index`).

1.2 bcd_wise_map

This table gives the filename of the WISE image used in SIMLA code for each separate BCD. There is one row for each DCEID.

Script: `/simlapath/database_build/bcd_wise_map.py`

Columns:

- DCEID: (int) code that is unique for each BCD.
- WISE_FILE: (str) for the WISE file path. Begins after the set `wisepath`, so it assumes that the WISE data directory structure has not been changed.

There are indices on DCEID and WISE_FILE (`bcd_wise_map_index`).

1.3 foreground_model

This table contains foreground model scaling information for each AOR. There is one row for each AORKEY. The data in this table is largely used for providing a proxy measurement of astrophysical foreground brightness when creating superdarks or for a zodi/ism cut in background construction. Estimates are given at $12\ \mu m$, corresponding to the fiducial wavelength of the WISE W3 band that is often used throughout SIMLA.

Script: `/simlapath/database_build/foreground_model.py`

Columns:

- AORKEY: (int) code that is unique for each observation area at a certain time.
- ISM_12: (float) an estimate in MJy/sr for the ISM foreground emission at $12\ \mu m$.
- ZODI_12: (float) an estimate in MJy/sr for the zodiacal light foreground emission at $12\ \mu m$.

There are indices on AORKEY, ISM_12, ZODI_12 (foreground_model.index).

1.4 judge1

This table contains “Judge 1” photometry for all shards in SL and LL observations. There is one row for each shard. Judge 1 photometry is used to judge whether a shard qualifies as a background observation, and to select dark observations to be included in superdarks. In this table, a value of 0.0 corresponds to NaN, and is typically filtered out.

Script: /simlapath/database_build/judge1.py or judge1_multip.py for parallelization.

Columns:

- DCEID: (int) code that is unique for each BCD.
- CHNLNUM: (int). SL, SH, LL, LH = 0, 1, 2, 3.
- SUBORDER: (int) either 1, 2, or 3, referring to SL1, SL2, SL3 or LL1, LL2, LL3.
- SHARD: (int) the ID number of a shard.
- BACKSUB.PHOT: (float) Judge 1 photometry value in MJy/sr.

There are indices on DCEID, CHNLNUM, SUBORDER, SHARD, and BACKSUB.PHOT (judge1.index).

1.5 judge2

This table contains “Judge 2” photometry for all shards in SL and LL observations. There is one row for each shard. Judge 2 photometry is used to judge whether a shard qualifies as a background observation.

Script: /simlapath/database_build/judge2.py or judge2_multip.py for parallelization.

Columns:

- DCEID: (int) code that is unique for each BCD.
- CHNLNUM: (int). SL, SH, LL, LH = 0, 1, 2, 3.
- SUBORDER: (int) either 1, 2, or 3, referring to SL1, SL2, SL3 or LL1, LL2, LL3.

- SHARD: (int) the ID number of a shard.
- SPECN: (float) a binned spectral value from the extracted subtracted spectrum on the BCD in MJy/sr. “N” is an integer, usually from 0-9, depending on the number of spectral bins. There is a corresponding column in this table for each spectral bin.
- F_MEDIAN: (float) the median value of the zodi and superdark subtracted Judge 2 spectral value on-order.
- F_MEAN: (float) the mean value of the zodi and superdark subtracted Judge 2 spectral value on-order.
- F_STDEV: (float) the standard deviation of values for the zodi and superdark subtracted Judge 2 spectrum on-order.

There are indices on DCEID, CHNLNUM, SUBORDER, SHARD, and F_MEDIAN (judge2.index).

NOTE: you can hand a pypika query output containing the full Judge 2 spectrum (SPEC0-SPECN) to `fmt_j2spec` in `simla_utils` for easy formatting.

1.6 shard_posositions

This table contains the sky coordinates in decimal degrees for the corners of shard apertures. There is one row for each shard.

Script: `/simlapath/database_build/shard_positions.py` or `shard_positions_multip.py` for parallelization.

Columns:

- DCEID: (int) code that is unique for each BCD.
- CHNLNUM: (int). SL, SH, LL, LH = 0, 1, 2, 3.
- SUBORDER: (int) either 1, 2, or 3, referring to SL1, SL2, SL3 or LL1, LL2, LL3.
- SHARD: (int) the ID number of a shard.
- C0_R: (float) RA decimal degree for corner 0 of this shard.
- C1_R: (float) RA decimal degree for corner 1 of this shard.
- C2_R: (float) RA decimal degree for corner 2 of this shard.
- C3_R: (float) RA decimal degree for corner 3 of this shard.
- C0_D: (float) DEC decimal degree for corner 0 of this shard.
- C1_D: (float) DEC decimal degree for corner 1 of this shard.
- C2_D: (float) DEC decimal degree for corner 2 of this shard.

- C3_D: (float) DEC decimal degree for corner 3 of this shard.

There are indices on DCEID, CHNLNUM, SUBORDER, and SHARD (shard_positions_index).
NOTE: you can hand a pypika query output containing the shard corners to `fmt_scorners` in `simla_utils` for easy formatting.

2 Script Run Order

Some database tables rely on other tables when building. Thus, the build order matters. Assuming the empty MySQL database has already been set up:

1. Copy and paste the commands in `database_init.txt`, stopping before `# INDICES` for now.
2. `init_scripts/trim_and_shard_masks.py`
3. `database_build/bcd_metadata.py`
4. `zodi_images/cvz_campaigns.py`
5. `zodi_images/make_zodi_images.py`
6. `database_build/bcd_wise_map.py`
7. `database_build/foreground_model.py`
8. `database_build/shard_positions.py` or `shard_positions_multip.py`
9. `database_build/judge1.py` or `judge1_multip.py`
10. `superdarks/make_superdarks.py`
11. `superdarks/make_tailored_superdarks.py`
12. `database_build/judge2.py` or `judge2_multip.py`
13. Create the database indices by copying and pasting the commands below `# INDICES` in `database_init.txt`.
14. Run the validation notebooks to make sure everything looks good.